

Project Report: HOMEFIND

Real Estate Property Listing & Viewing Scheduler Application

Name: Viketh Hegde

Roll No: 150076924071

Cohort: Mark Zuckerberg

1. EXECUTIVE SUMMARY & SYSTEM VISION

1.1 Project Overview & Value Proposition

HomeFind is an enterprise-grade, cross-platform real estate property listing and viewing appointment scheduling application built with Flutter and Firebase. The application bridges the operational gap between real estate agents and property buyers by providing end-to-end digital property discovery, automated viewing slot management, and real-time appointment reservation.

1.2 Solving Modern Real Estate Friction

Traditional real estate platforms suffer from major operational bottlenecks:

- **Outdated Property Information:** Static listing portals suffer from delayed updates when properties are sold, rented, or delisted, leading to buyer frustration and agent inefficiency.
- **Manual Viewing Coordination:** Organizing viewing appointments usually requires phone calls, back-and-forth messaging, or manual spreadsheet maintenance, consuming valuable agent hours.
- **Double-Booking & Race Conditions:** High-demand properties often suffer from overlapping appointment bookings when multiple buyers attempt to reserve the same time window simultaneously.

HomeFind resolves these issues through:

1. **Real-Time Data Synchronization:** Instant interface updates across all connected clients using Cloud Firestore streams.
2. **Automated Slot Scheduling:** Agents generate dynamic time slots; buyers pick and reserve slots directly.

- 3. **Atomic Concurrency Control:** Firestore database transactions prevent concurrent reservation of the exact same slot.

1.3 Dual-Role Architecture

HomeFind enforces strict role-based operational segregation between two principal user roles:

•

Feature / Capability	Property Buyer	Real Estate Agent
Authentication & Profile	Role: buyer	Role: agent
Property Discovery	Browse, full-text search, category filter	View own published listings
Listing Management	Read-only access to property details	Create, edit, and delete property listings
Slot Management	View available date & time slots	Generate and manage viewing slot availability
Appointment Booking	Atomic slot booking, view scheduled visits	Monitor incoming bookings across listings
Cancellation Workflow	Cancel appointment (releases slot)	Manage and review appointment statuses

2. SYSTEM ARCHITECTURE & MODULAR LAYERED DESIGN

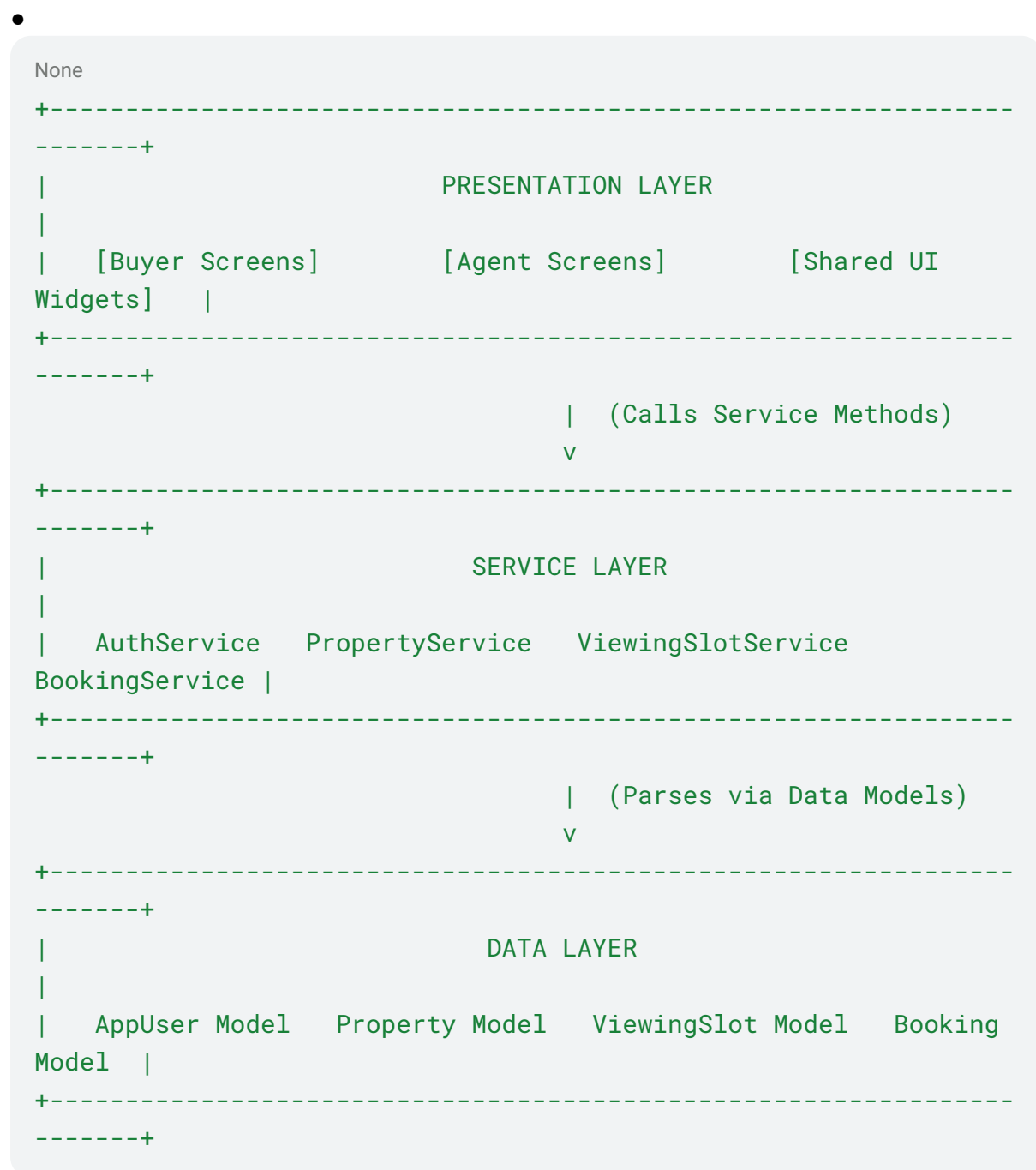
2.1 3-Tier Layered Architecture

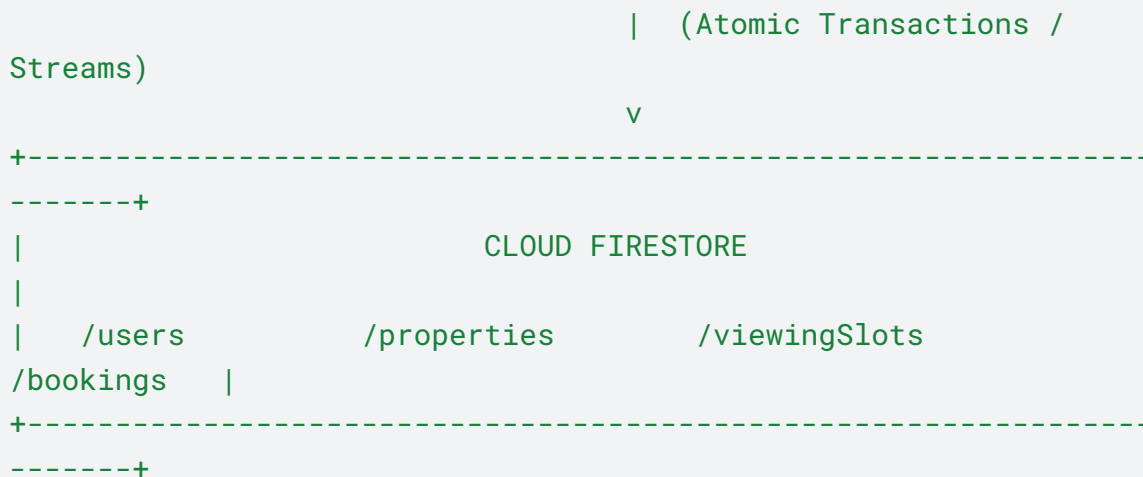
HomeFind adheres to a clean, modular 3-tier architecture ensuring strict Separation of Concerns (SoC) and maintainability:

- 1. **Presentation Layer (lib/screens/, lib/widgets/, lib/theme/):**
 - 2. Renders responsive Material Design 3 UI components.
 - 3. Uses StreamBuilder and FutureBuilder to reactively render database changes without local state bloat.
 - 4. Strictly decoupled from direct database drivers; UI widgets call service methods exclusively.
- 5. **Service Layer (lib/services/):**
 - 6. Encapsulates all core business logic, Firebase Authentication calls, Cloud Firestore operations, and transactional workflows.

7. Includes AuthService, PropertyService, ViewingSlotService, BookingService, and AssetImageService.
8. **Data Layer (lib/models/):**
 9. Defines strongly typed immutable Dart domain models (AppUser, Property, ViewingSlot, Booking).
 10. Provides standardized toMap() serialization and fromFirestore() / fromMap() deserialization factories.

2.2 Architectural Data Flow Diagram





2.3 Architectural Design Patterns

- **Encapsulation / Service Pattern:** Database queries and mutations are isolated in service classes. Screens never execute raw Firestore queries.
- **Factory Method Pattern:** Standardized model instantiation from Cloud Firestore document snapshots (`fromFirestore`).
- **Reactive Streams Pattern:** UI components listen directly to Firestore snapshots(), establishing single-source-of-truth reactivity.

3. DATABASE SCHEMA & FIRESTORE ARCHITECTURE

Cloud Firestore is configured as a NoSQL document database comprising four top-level collections:

-

```

None
Cloud Firestore Root
├─ users/ (Collection)
├─ properties/ (Collection)
├─ viewingSlots/ (Collection)
└─ bookings/ (Collection)

```

3.1 users Collection Schema

Document ID: {uid} (Matches Firebase Auth UID)

-

JSON

```
{
  "uid": "STRING (Primary Key, Auth UID)",
  "name": "STRING",
  "email": "STRING",
  "role": "STRING ('buyer' | 'agent')",
  "createdAt": "TIMESTAMP"
}
```

3.2 properties Collection Schema

Document ID: {propertyId} (Auto-generated UUID/Firestore ID)

-

JSON

```
{
  "id": "STRING (Document ID)",
  "title": "STRING",
  "description": "STRING",
  "price": "NUMBER (Double/Int in USD/Local Currency)",
  "area": "NUMBER (Square feet/meters)",
  "location": "STRING (City, Neighborhood, Address)",
  "bedrooms": "INTEGER",
  "bathrooms": "INTEGER",
  "propertyType": "STRING ('Apartment' | 'Villa' | 'Penthouse' | 'Studio' | 'Townhouse')",
  "primaryImage": "STRING (Asset image path / Storage URL)",
  "images": "ARRAY<STRING> (Asset image paths / Storage URLs)",
  "agentId": "STRING (Foreign Key -> users.uid)",
  "createdAt": "TIMESTAMP"
}
```

3.3 viewingSlots Collection Schema

Document ID: {slotId} (Auto-generated UUID/Firestore ID)

-

JSON

```
{
  "id": "STRING (Document ID)",
  "propertyId": "STRING (Foreign Key -> properties.id)",
  "date": "STRING ('YYYY-MM-DD')",
  "startTime": "STRING ('HH:mm')",
  "endTime": "STRING ('HH:mm')",
  "isAvailable": "BOOLEAN (true = Open, false = Reserved)"
}
```

3.4 bookings Collection Schema

Document ID: {bookingId} (Auto-generated UUID/Firestore ID)

-

JSON

```
{
  "id": "STRING (Document ID)",
  "propertyId": "STRING (Foreign Key -> properties.id)",
  "slotId": "STRING (Foreign Key -> viewingSlots.id)",
  "propertyTitle": "STRING (Denormalized for performance)",
  "buyerName": "STRING",
  "buyerEmail": "STRING",
  "date": "STRING ('YYYY-MM-DD')",
  "startTime": "STRING ('HH:mm')",
  "endTime": "STRING ('HH:mm')",
  "status": "STRING ('confirmed' | 'cancelled')",
  "createdAt": "TIMESTAMP"
}
```

4. CONCURRENCY CONTROL & ATOMIC BOOKING ENGINE

4.1 The Race Condition Challenge

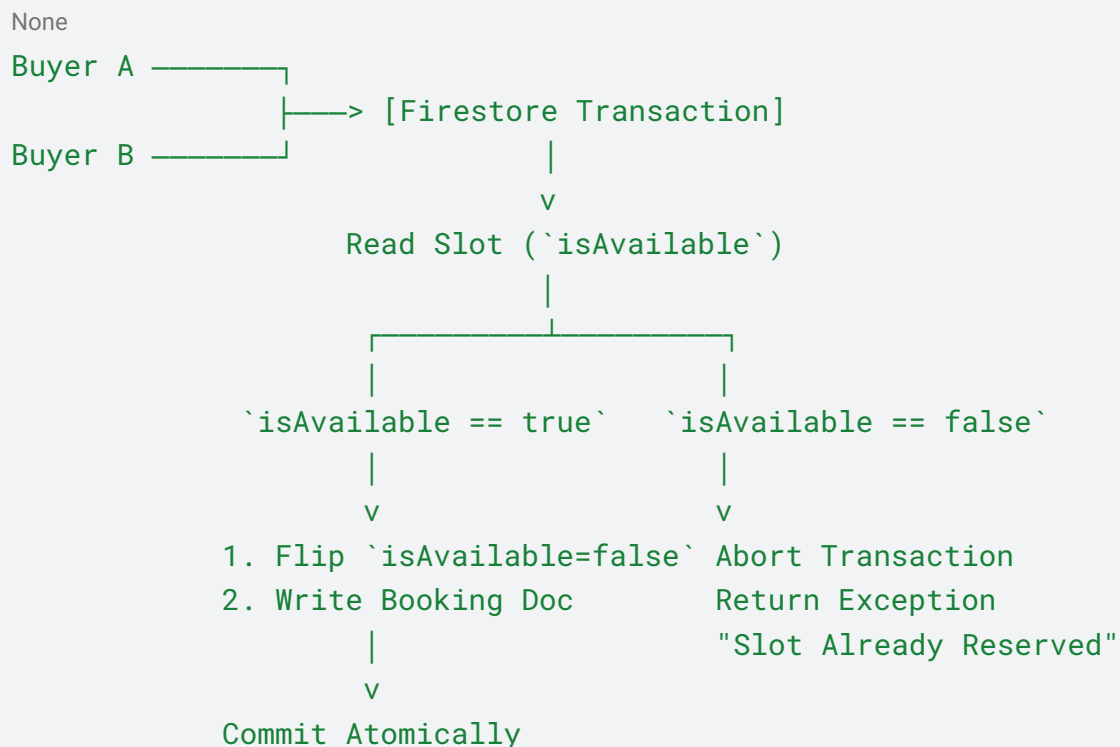
When two buyers simultaneously open the booking screen for a high-demand property and attempt to book the exact same time slot (`slotId`), standard non-atomic read-then-write logic causes a race condition:

11. Buyer A reads `isAvailable == true`.
12. Buyer B reads `isAvailable == true` before Buyer A writes back.
13. Buyer A writes `isAvailable = false` and creates Booking A.
14. Buyer B writes `isAvailable = false` and creates Booking B.
15. **Result:** Double-booking occurred; two appointments exist for one slot.

4.2 Atomic Firestore Transactions Solution

HomeFind guarantees 100% data integrity using `FirestoreTransaction.runTransaction()`. Firestore transactions ensure that all read operations happen before write operations and lock the target document snapshot. If another client updates the target document during transaction execution, Firestore automatically retries the entire transaction block.

•



4.3 Production Dart Implementation Flow

-

Dart

```
class BookingService {
  final FirebaseFirestore _db = FirebaseFirestore.instance;

  Future<void> bookViewingSlot({
    required String slotId,
    required String propertyId,
    required String propertyTitle,
    required String buyerName,
    required String buyerEmail,
    required String date,
    required String startTime,
    required String endTime,
  }) async {
    final DocumentReference slotRef =
      _db.collection('viewingSlots').doc(slotId);
    final DocumentReference bookingRef =
      _db.collection('bookings').doc();

    return _db.runTransaction((transaction) async {
      // 1. Read the slot document within the transaction
      final DocumentSnapshot slotSnapshot = await
transaction.get(slotRef);

      if (!slotSnapshot.exists) {
        throw Exception("The requested viewing slot no longer
exists.");
      }

      final bool isAvailable = slotSnapshot.get('isAvailable') ??
false;

      // 2. Check slot availability
      if (!isAvailable) {
        throw Exception("Slot Unavailable: This slot was just
reserved by another user.");
      }
    });
  }
}
```

```

// 3. Mark the slot as unavailable
transaction.update(slotRef, {'isAvailable': false});

// 4. Create the new booking document atomically
transaction.set(bookingRef, {
    'id': bookingRef.id,
    'propertyId': propertyId,
    'slotId': slotId,
    'propertyTitle': propertyTitle,
    'buyerName': buyerName,
    'buyerEmail': buyerEmail,
    'date': date,
    'startTime': startTime,
    'endTime': endTime,
    'status': 'confirmed',
    'createdAt': FieldValue.serverTimestamp(),
});
});
}

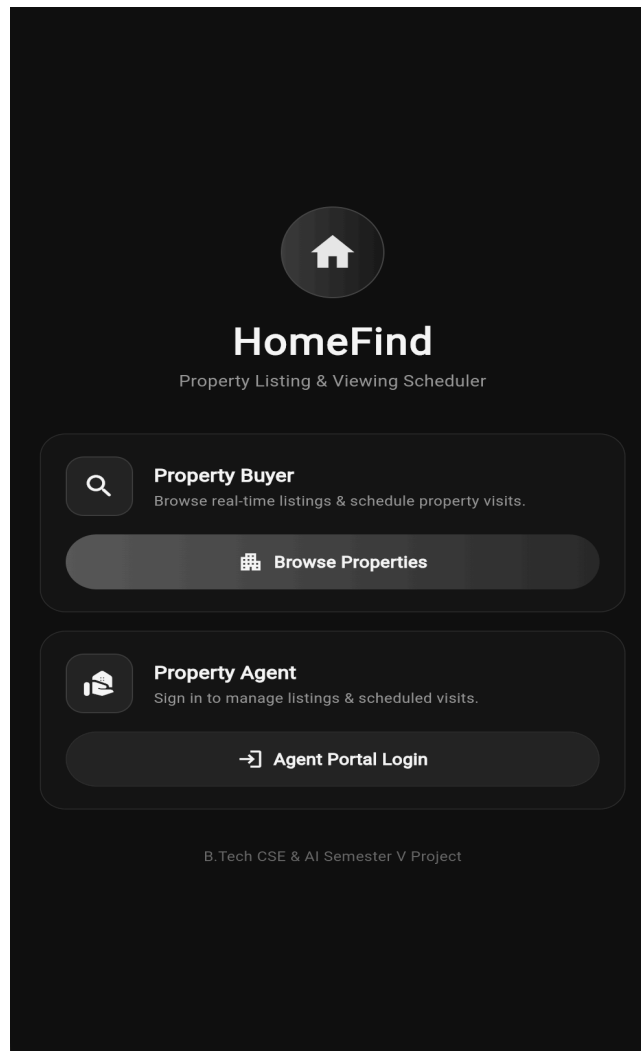
// Appointment Cancellation Workflow (Releases Slot)
Future<void> cancelBooking({
    required String bookingId,
    required String slotId,
}) async {
    final DocumentReference bookingRef =
_db.collection('bookings').doc(bookingId);
    final DocumentReference slotRef =
_db.collection('viewingSlots').doc(slotId);

    return _db.runTransaction((transaction) async {
        transaction.update(bookingRef, {'status': 'cancelled'});
        transaction.update(slotRef, {'isAvailable': true});
    });
}
}

```

5. USER INTERFACE, UX, & RESPONSIVE DESIGN SYSTEM

5.1 Material Design 3 Dark Theme Specifications



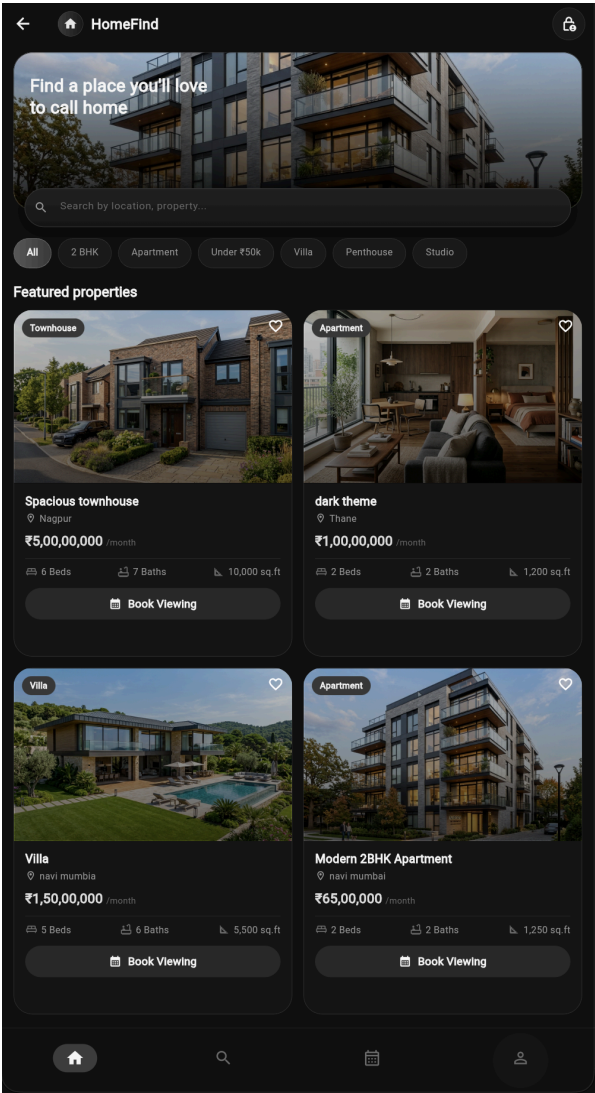
HomeFind uses a dark obsidian Material Design 3 theme defined in `lib/theme/app_theme.dart`:

- **Color Palette:**
 - `scaffoldBackgroundColor`: Deep Obsidian (#121212)
 - `cardColor`: Charcoal Surface (#1E1E1E)
 - `primary`: Clean White Accent (FFFFFF)
 - `secondary`: Muted Silver Text (#A0A0A0)
 - `border`: Subdued Dark Border (#2C2C2C)
- **Typography**: Google Fonts Inter / Roboto with sharp contrast hierarchical headings.
- **Component Styling**: Card elevation 0 with 12px border radius and 1px custom outline stroke.

5.2 Multi-Platform Responsive Layout Strategy

- HomeFind utilizes LayoutBuilder to dynamic adapt layout grid columns and aspect ratios across screen form factors:

Screen Breakpoint	Target Form Factor	Column Grid Count	Aspect Ratio	UX Layout Strategy
< 550 px	Mobile Portrait	1 Column	1.1 - 1.2	Full-width stacked cards, single-column scroll
550 – 749 px	Large Mobile / Tablet Port.	2 Columns	0.95 - 1.05	Two-column grid with compact filter bar
750 – 1099 px	Tablet Landscape	3 Columns	0.85 - 0.95	Multi-column grid, split-pane property details
>= 1100 px	Desktop / Web	4 Columns	0.80 - 0.90	Wide-screen grid with fixed side navigation panel



6. CORE FUNCTIONAL MODULES & WORKFLOWS

6.1 Authentication Module

- **Firebase Auth Integration:** Email/Password authentication.
- **Role-Based Routing:** Upon authentication, AuthService fetches user role from `/users/{uid}`.
 - If `role == 'agent'`, route to `AgentDashboardScreen`.

[Insert Screenshot Here: Authentication Flow & User Sign-In Screen]

- If `role == 'buyer'`, route to `BuyerHomeScreen`.

6.2 Property Management & Discovery Module

- **Real-time Browsing:** Utilizes `StreamBuilder<QuerySnapshot>` to stream properties ordering by `createdAt desc`.
- **Category Filtering:** Instant client-side/Firestore query filtering across types (Apartment, Villa, Penthouse, Studio, Townhouse).

[Insert Screenshot Here: Property Browsing, Category Filtering, and Search Interface]

- **Full-Text Search:** Local case-insensitive matching on property title and location strings.

6.3 Viewing Slot & Booking Module

- **Slot Generation (Agents):** Agents pick property ID, date, start time, and end time to write `/viewingSlots` documents.
- **Slot Selection & Booking (Buyers):** Interactive date picker calendar filtering open slots (`isAvailable == true`).

[Insert Screenshot Here: Viewing Slot Calendar Picker and Booking Confirmation Screen]

- **Cancellation Flow:** Atomically sets booking status to cancelled and flips slot `isAvailable` back to true.
-

7. SECURITY RULES, VALIDATION, & ERROR HANDLING

7.1 Cloud Firestore Security Rules

JavaScript

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper functions
    function isAuthenticated() {
      return request.auth != null;
    }

    function isOwner(userId) {
      return isAuthenticated() && request.auth.uid == userId;
    }

    function getUserRole() {
      return
get(/databases/{database}/documents/users/{request.auth.uid}).data.role;
    }

    function isAgent() {
      return isAuthenticated() && getUserRole() == 'agent';
    }

    // Users Collection
    match /users/{userId} {
      allow read: if isAuthenticated();
      allow create: if isAuthenticated() && request.auth.uid ==
userId;
      allow update, delete: if isOwner(userId);
    }

    // Properties Collection
    match /properties/{propertyId} {
```

```

        allow read: if isAuthenticated();
        allow create: if isAgent() && request.resource.data.agentId
== request.auth.uid;
        allow update, delete: if isAgent() && resource.data.agentId
== request.auth.uid;
    }

    // Viewing Slots Collection
    match /viewingSlots/{slotId} {
        allow read: if isAuthenticated();
        allow create, update, delete: if isAgent();
    }

    // Bookings Collection
    match /bookings/{bookingId} {
        allow read: if isAuthenticated();
        allow create: if isAuthenticated();
        allow update: if isAuthenticated();
    }
}
}
}

```

7.2 Centralized Input Validation & Error Handling

- **Validators Utility Class:**
 - Email regex format validation.
 - Password length requirement (≥ 6 chars).
 - Numeric & positive checks for price, bedrooms, bathrooms, and area dimensions.
- **ErrorHandler Utility Class:**
 - Converts raw FirebaseAuthException codes (user-not-found, wrong-password, email-already-in-use) into friendly, localized user notification snackbars.

Handles Firestore FirebaseException exceptions smoothly during transaction failures.